Image Generation Using Variational Autoencoder (VAE)

```
Warning: You are sending unauthenticate import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, Subset
import matplotlib.pyplot as plt

# AI23531 Deep Learning Page | 32

# Step 2: Configure Parameters and Load Dataset
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
image_size = 28
batch_size = 128
latent_dim = 20
num_epochs = 5
learning_rate = 1e-3

transform = transforms.Compose([
    transforms.Resize(image_size),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Replaced CelebA with MNIST
dataset = datasets.MNIST(root='data', train=True, download=True,
transform=transform)
dataset = Subset(dataset, range(500)) # Limit to 500 samples for speed
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

# Step 3: Define Encoder Network
class Encoder(nn.Module):
    def __init__(self, latent_dim):
        super(Encoder, self).__init__()
        # MNIST has 1 input channel instead of 3
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, 2, 1), nn.ReLU(),            # Output: 32 x 14 x
14
            nn.Conv2d(32, 64, 3, 2, 1), nn.BatchNorm2d(64), nn.ReLU(),  # Output:
64 x 7 x 7
        )
        self.fc_mu = nn.Linear(64 * 7 * 7, latent_dim)
```

```
        self.fc_logvar = nn.Linear(64 * 7 * 7, latent_dim)

    def forward(self, x):
        x = self.conv(x)
        x = x.view(x.size(0), -1)
        return self.fc_mu(x), self.fc_logvar(x)

# Step 4: Define Decoder Network
class Decoder(nn.Module):
    def __init__(self, latent_dim):
        super(Decoder, self).__init__()
        self.fc = nn.Linear(latent_dim, 64 * 7 * 7)
        self.deconv = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 3, 2, 1, output_padding=1),
nn.BatchNorm2d(32), nn.ReLU(), # Output: 32 x 14 x 14
            nn.ConvTranspose2d(32, 1, 3, 2, 1, output_padding=1), nn.Tanh()
# Output: 1 x 28 x 28
        )

    def forward(self, z):
        x = self.fc(z)
        x = x.view(x.size(0), 64, 7, 7)
        return self.deconv(x)

# Step 5: Define the VAE Model
class VAE(nn.Module):
    def __init__(self, latent_dim):
        super(VAE, self).__init__()
        self.encoder = Encoder(latent_dim)
        self.decoder = Decoder(latent_dim)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def forward(self, x):
        mu, logvar = self.encoder(x)
        z = self.reparameterize(mu, logvar)
        return self.decoder(z), mu, logvar

# Step 6: Define Loss Function
def vae_loss(recon_x, x, mu, logvar):
```

```
    recon_loss = F.mse_loss(recon_x, x, reduction='sum')
    kl_div = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon_loss + kl_div

# Step 7: Train the Model and Generate Images
model = VAE(latent_dim).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

for epoch in range(num_epochs):
    model.train()
    total_loss = 0
    for images, _ in dataloader:
        images = images.to(device)
        recon, mu, logvar = model(images)
        loss = vae_loss(recon, images, mu, logvar)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    print(f"Epoch [{epoch+1}/{num_epochs}], Loss:
{total_loss/len(dataloader.dataset):.4f}")

# Generate and visualize samples
model.eval()
with torch.no_grad():
    z = torch.randn(64, latent_dim).to(device)
    sample_images = model.decoder(z).cpu() * 0.5 + 0.5 # De-normalize
    grid = torchvision.utils.make_grid(sample_images, nrow=8)
    plt.imshow(grid.permute(1, 2, 0).squeeze(), cmap='gray')
    plt.axis('off')
    plt.title(f"Generated Digits - Epoch {epoch+1}
```

OUTPUT:

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 55.3MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 6.03MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 14.9MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 9.40MB/s]
```

```
Epoch [1/5], Loss: 614.7824
Epoch [2/5], Loss: 406.2737
Epoch [3/5], Loss: 305.5581
Epoch [4/5], Loss: 267.7359
Epoch [5/5], Loss: 246.9213
```

